Assignment 3: Parallel Matrix Multiplication and Scalability Analysis

Big Data - Degree in Data Science and Engineering (ULPGC)

Author: Sara Dévora Ortega | **Date:** 11 November 2026

Abstract

This study evaluates the shared-memory parallel scalability of dense matrix multiplication using C and OpenMP across thread counts ($p \in \{1, 2, 4, 8, 16, 24\}$) and matrix dimensions ($n \in [10, 1000]$). Executed on an 8-core / 16-thread Intel Core i7-11800H processor, an optimized sequential *I-K-J* baseline (T_{seq}) is evaluated alongside OpenMP iterations (T_p). At small dimensions ($n \leq 50$), thread startup overhead causes parallel execution to degrade performance relative to the unthreaded baseline. For large matrices ($n = 1000$), parallel scaling demonstrates high efficiency up to physical cores ($S(4) = 3.62\times, E(4) = 0.90$), reaching a performance plateau around $\sim 197 - 201$ ms for $p \geq 8$ ($S \approx 4.15 - 4.22\times$). Empirical analysis shows that shared DDR4 memory bandwidth saturation bounds speedup past 8 threads, while oversubscription ($p = 24$) reduces efficiency ($E = 0.18$) without causing runtime catastrophies.

1 Introduction and Research Objectives

Shared-memory multi-threading with OpenMP allows loop iterations to be partitioned across CPU cores. Building upon the cache-friendly *I-K-J* algorithm from Assignment 2, this investigation addresses performance scaling, thread overhead, and hardware limits.

1. **Primary Research Question:** How do thread count p and matrix size n impact execution time $T_p(n)$, Speedup $S(p)$, and Efficiency $E(p)$, and at what matrix size does parallelization overcome thread management overhead?
2. **Secondary Research Question:** What performance behavior occurs when scaling from physical cores ($p \leq 8$) to logical processors via Hyper-Threading ($p = 16$) and software oversubscription ($p = 24$)?

2 Methodology and Experimental Protocol

2.1 Hardware and Toolchain Context

- **Processor:** Intel Core i7-11800H (8 physical cores, 16 logical threads via Hyper-Threading, 2.30 GHz base). Per-core L1d cache: 48 KB, L2 cache: 1.25 MB per core, L3 shared cache: 24 MB.
- **Memory & System:** 16 GB DDR4 RAM, Windows 11 Home 64-bit.
- **Compiler & Flags:** GCC 13.2.0 (w64devkit distribution) compiled via `gcc -O2 -fopenmp partC_parallel.c -o partC_parallel.exe`.

2.2 Work Sharing and Correctness

The outer i-loop is parallelized using static scheduling (`#pragma omp parallel for schedule(static)`). Data access safety is guaranteed as each thread writes exclusively to distinct rows of C. Matrix elements were generated with a fixed seed (`srand(42)`). Numerical correctness was verified across all modes by comparing checksums ($C_{0,0} + C_{n-1,n-1}$), yielding exact matches across all runs.

2.3 Benchmarking Protocol

High-resolution timers (`QueryPerformanceCounter`) isolate kernel execution. Each configuration (n, p) was measured for 5 repetitions, recording the median runtime and Interquartile Range (IQR $= Q_3 - Q_1$). Both raw iteration files (`results_partC_raw.csv`) and summary datasets (`results_partC.csv`) were exported.

3 Results and Performance Analysis

Table 1 presents the empirical median runtime and IQR across representative matrix sizes $n \in \{10, 100, 500, 800, 1000\}$.

Table 1: Empirical Execution Times (Median ms ± IQR) Across Matrix Sizes

Configuration	$n = 10$	$n = 100$	$n = 500$	$n = 800$	$n = 1000$
Sequential Baseline	**0.001** ± 0.001	1.27 ± 0.02	76.13 ± 8.74	354.01 ± 182.53	835.71 ± 129.68
OpenMP $p = 1$	0.101 ± 0.031	1.37 ± 0.13	73.63 ± 13.76	300.11 ± 28.71	694.33 ± 64.69
OpenMP $p = 2$	0.492 ± 0.069	1.01 ± 0.22	65.59 ± 20.25	173.37 ± 17.13	409.76 ± 14.61
OpenMP $p = 4$	0.429 ± 0.050	1.00 ± 0.23	35.25 ± 13.21	133.28 ± 13.25	231.02 ± 8.26
OpenMP $p = 8$	1.257 ± 0.337	1.39 ± 0.67	**28.02** ± 12.75	**105.59** ± 21.63	**199.10** ± 17.81
OpenMP $p = 16$	2.337 ± 0.454	2.47 ± 0.60	36.68 ± 13.01	107.14 ± 11.32	201.33 ± 17.30
OpenMP $p = 24$	3.137 ± 1.343	3.68 ± 1.07	36.92 ± 1.80	105.68 ± 24.95	197.77 ± 12.20

Table 2 outlines the derived Speedup $S(p) = T_{\text{seq}}/T_p$ and Efficiency $E(p) = S(p)/p$ for $n = 1000$ ($T_{\text{seq}} = 835.71$ ms).

Table 2: Parallel Scaling Metrics for $n = 1000$ ($T_{\text{seq}} = 835.71$ ms)

Threads (p)	**Median Time (ms)**	**IQR (ms)**	**Speedup** $S(p)$	**Efficiency** $E(p)$
1 (Seq Baseline)	835.71	129.68	1.00	1.00
1 (OpenMP)	694.33	64.69	1.20	1.20
2	409.76	14.61	2.04	1.02
4	231.02	8.26	3.62	0.90
8	199.10	17.81	4.20	0.52
16	201.33	17.30	4.15	0.26
24 (Oversubscribed)	197.77	12.20	4.23	0.18

4 Discussion and Limitations

4.1 Threshold of Parallel Utility

At small sizes ($n = 10, 50$), sequential execution is significantly faster (0.0013 ms at $n = 10$) than parallel execution ($0.429 - 3.137$ ms). The overhead of OpenMP thread management and synchronization dominates small workloads. Parallelization only becomes beneficial when $n \geq 100$, where compute intensity offsets thread initialization costs.

4.2 Physical Core Scaling and Memory Bandwidth Plateau

For $n = 1000$, strong linear scaling occurs up to 4 threads ($S(4) = 3.62\times, E(4) = 0.90$). At $p = 2$, efficiency reaches 1.02, which is explained by slight baseline measurement noise (IQR = 129.68 ms) and effective per-core L1/L2 cache capacity partitioning.

Past $p = 4$, performance enters a distinct **plateau** around $\sim 197 - 201$ ms ($S \approx 4.15 - 4.23\times$). The runtimes for $p = 8$ (199.10 ms), $p = 16$ (201.33 ms), and $p = 24$ (197.77 ms) are statistically

indistinguishable, as their variations fall well within the measured IQR ($\sim 12 - 17$ ms). This behavior indicates that execution speed becomes bound by shared DDR4 memory bus bandwidth rather than raw compute capacity.

4.3 Oversubscription and Hyper-Threading Impact

Hyper-Threading ($p = 16$) and oversubscription ($p = 24$) do not cause catastrophic execution slowdowns; however, parallel efficiency declines rapidly to $E(16) = 0.26$ and $E(24) = 0.18$ simply because additional threads provide no speedup once memory bandwidth is saturated.

4.4 Limitations and Threats to Validity

- **System Noise:** Background Windows operating system scheduling introduces non-negligible variance in short runs, reflected in the baseline IQR.
- **Auto-Vectorization:** The compiler flag `-O2` allows GCC to generate SIMD instructions automatically, meaning timing reflects combined thread-level parallelism and instruction-level vectorization.

5 Conclusions

Parallel execution using C and OpenMP accelerates dense matrix multiplication for matrices with $n \geq 100$. Near-linear speedups are achieved up to $p = 4$ threads ($3.62\times$). Beyond 4 threads, execution hits a memory bandwidth saturation ceiling at $\sim 4.2\times$ speedup, rendering higher thread counts ($p = 8, 16, 24$) inefficient without incurring execution runtime penalties.

Materials and AI Declaration

Reproducibility: Source code, raw execution measurements, and summary CSVs are available at: https://github.com/Saradevora/Assignment-3.

AI Declaration: Generative AI tools were used to assist in data processing scripts

References

[1] L. Dagum and R. Menon, "OpenMP: an industry standard API for shared-memory programming," *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.

[2] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Pearson, 2015.